NODE.JS BACKEND
MASTERY GUIDE

From Zero to God-Level Backend Engineer

22 Chapters | 300+ Code Examples | 150+ Interview Q&A | System Design
One Mega Project: NEXUS — AI-Powered Distributed API Gateway

Prepared for: Pratik Gond | NIT Kurukshetra | 90-Day Placement Prep | 2026

THE PROJECT: NEXUS — AI-Powered Distributed API Gateway

What We Are Building

NEXUS is a production-grade, AI-powered distributed API Gateway and Developer Platform. Think of it as building your own Cloudflare Workers + Kong API Gateway + OpenAI integration — from scratch. This is NOT a tutorial project. This is the kind of system that runs at companies like Stripe, Cloudflare, and AWS. By the time you finish, you will have built something that NO other fresher has on their resume.

NEXUS Architecture Overview

NEXUS has 7 major subsystems, each mapping to topics in this guide:

Subsystem	What It Does	Tech Stack	Topics You'll Learn
1. Gateway Core	Receives HTTP requests, routes to upstream services, transforms req/res	Node.js, Express, http module	Ch 1-5: Node internals, HTTP, Express, Middleware
2. Auth Server	OAuth2 + JWT auth, API key management, RBAC, rate limiting per user	JWT, bcrypt, Redis, PostgreSQL	Ch 6-8: Auth, Databases, ORM
3. Plugin Engine	Extensible plugin system — add custom middleware (logging, transform, cache)	EventEmitter, Dynamic imports	Ch 9-10: API Design, Plugin Architecture
4. Real-Time Dashboard	Live metrics, WebSocket streaming, connection monitoring	Socket.io, Redis Pub/Sub	Ch 11-12: WebSockets, Message Queues
5. AI Layer	LLM-powered log analysis, anomaly detection, auto-generate API docs	OpenAI API, Embeddings, RAG	Ch 20: AI Integration
6. Distributed Core	Multi-instance coordination, consistent hashing, health checks	Cluster, Worker Threads, Docker	Ch 13-14, 21: Docker, Scaling
7. Observability	Structured logging, distributed tracing, alerting, performance metrics	Winston, Prometheus, Grafana	Ch 17-18: Monitoring, CI/CD

Why This Project Will Blow Interviewers' Minds

1) Nobody builds an API Gateway from scratch — it shows you understand networking at a deep level. 2) The AI integration (auto-anomaly detection, auto-documentation) is cutting-edge and shows GenAI skills. 3) The plugin architecture shows you think about extensibility — a senior-engineer mindset. 4) Multi-instance coordination shows you understand distributed systems. 5) Every subsystem maps directly to interview topics. When asked 'tell me about a project,' you have 30+ minutes of deep technical content.

How This Guide Works

Each chapter teaches a topic with theory, code, and interview Q&A. At the end of chapters that contribute to NEXUS, you'll get a BUILD TASK — a specific piece of NEXUS to implement using what you just learned. No solutions given — only hints. By Chapter 22, all pieces connect into the complete NEXUS system.

Table of Contents

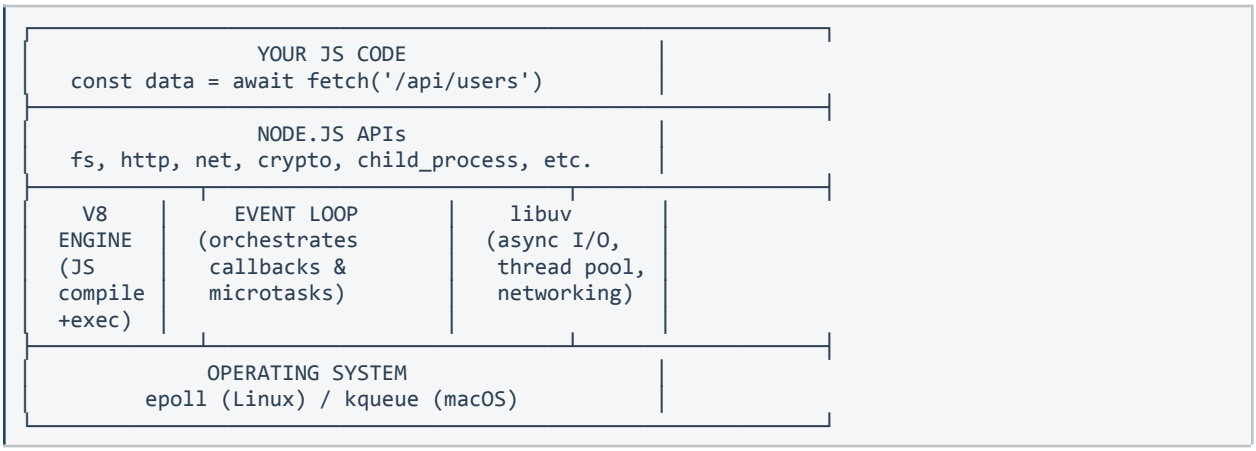
Chapter 1: Node.js Internals — Event Loop, V8, libuv

NEXUS Relevance: *CRITICAL — Understanding the event loop is why NEXUS can handle 10,000+ concurrent connections on a single thread.*

1.1 What is Node.js? (The Real Answer)

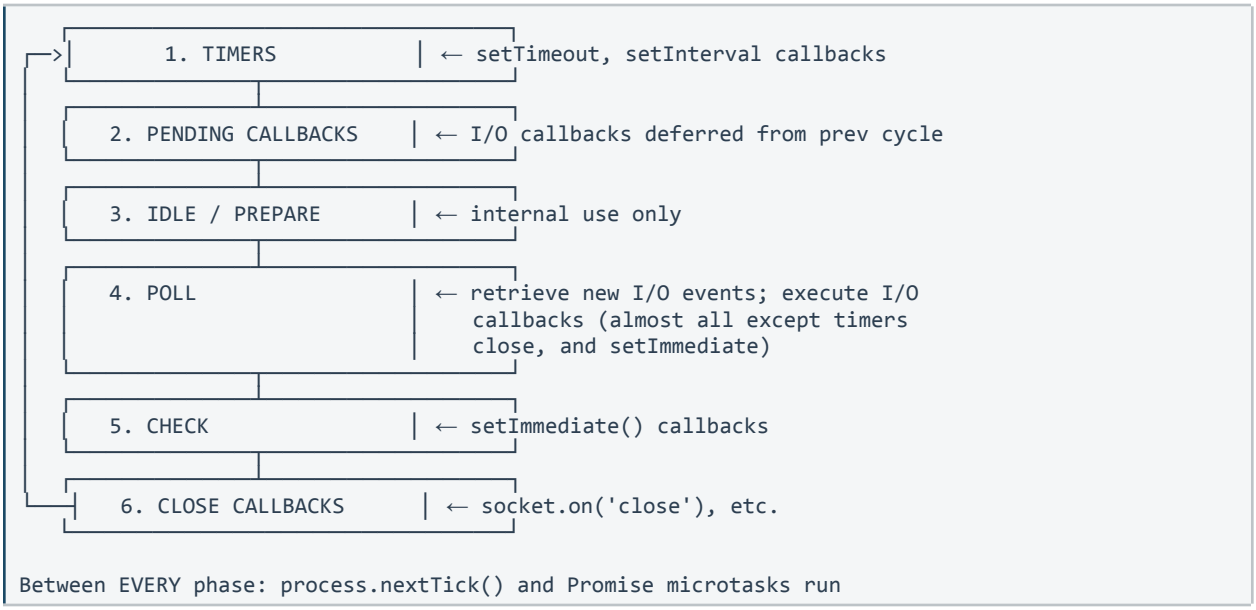
Node.js is NOT a language, NOT a framework. It is a JavaScript runtime built on Chrome's V8 engine, combined with libuv (a C library for async I/O). The key insight: Node.js runs JavaScript in a single thread but handles I/O operations asynchronously through an event loop and a thread pool.

Architecture Diagram (Mental Model)



1.2 The Event Loop — 6 Phases (Most Misunderstood Topic)

The event loop is NOT a simple queue. It has 6 distinct phases, each with its own queue. Understanding the order is critical for debugging and interviews.



The Trick Question: What's the Output?

```
console.log('1: script start');

setTimeout(() => console.log('2: setTimeout'), 0);

Promise.resolve().then(() => console.log('3: promise'));

process.nextTick(() => console.log('4: nextTick'));

setImmediate(() => console.log('5: setImmediate'));

console.log('6: script end');

// OUTPUT:
// 1: script start
// 6: script end
// 4: nextTick     ← nextTick runs BEFORE promises
// 3: promise      ← microtask queue (after nextTick)
// 2: setTimeout   ← timer phase
// 5: setImmediate ← check phase (after poll)
```

Why This Matters for NEXUS

NEXUS processes thousands of API requests per second. Each request triggers I/O (database queries, upstream HTTP calls). The event loop ensures these don't block each other. A single slow synchronous operation (like JSON.parse on a 100MB payload) would freeze ALL concurrent requests. Understanding this prevents catastrophic performance bugs.

1.3 V8 Engine — How JavaScript Actually Executes

V8 compiles JavaScript to machine code in two stages:

1. Ignition (Interpreter): Quickly generates bytecode. Fast startup but slower execution. Used for code that runs once.

2. TurboFan (Optimizing Compiler): Watches 'hot' functions (called many times). Compiles them to highly optimized machine code. This is why Node.js gets faster over time for hot paths.

3. Deoptimization: If assumptions TurboFan made break (e.g., a function that always received numbers suddenly gets a string), it falls back to Ignition. This is called 'deopt' and causes a performance cliff.

Hidden Classes & Inline Caching (Interview Gold)

```
// BAD: Adding properties in different orders creates different
// hidden classes, preventing V8 optimization
function Point(x, y) {
  if (x) this.x = x;  // Sometimes x is set first
  if (y) this.y = y;  // Sometimes y is set first
}

// GOOD: Always initialize properties in the same order
function Point(x, y) {
  this.x = x;  // Always x first
  this.y = y;  // Always y second
}

// TERRIBLE: Adding properties dynamically
const obj = {};
obj.a = 1;  // hidden class transition
obj.b = 2;  // another transition
obj.c = 3;  // another transition
```

```
// Each transition creates a new hidden class chain

// BETTER: Define all properties upfront
const obj = { a: 1, b: 2, c: 3 };  // single hidden class
```

1.4 Memory Management & Garbage Collection

V8 divides the heap into two spaces:

Young Generation (New Space): Small (~1-8MB). Objects are created here. Scavenged (minor GC) frequently. Most objects die young (90%+) and are collected here. Very fast.

Old Generation (Old Space): Large (~700MB-1.5GB default). Objects that survive 2+ scavenges get promoted here. Collected via Mark-Sweep-Compact (major GC). Slower — causes pauses.

Memory Leak Detection (Critical for Production)

```
// Common memory leak patterns in Node.js:

// 1. CLOSURES holding references
function createLeak() {
  const hugeArray = new Array(1000000).fill('x');
  return function() {
    // This closure keeps hugeArray alive forever
    return hugeArray.length;
  };
}
const leak = createLeak(); // hugeArray can never be GC'd

// 2. EVENT LISTENERS not removed
const EventEmitter = require('events');
const emitter = new EventEmitter();
function handler() { /* ... */ }
// Adding listener in a loop without removing = leak
setInterval(() => {
  emitter.on('data', handler); // LEAK: adds new listener every time
}, 1000);

// 3. GLOBAL CACHES without eviction
const cache = {};
app.get('/api/:id', (req, res) => {
  cache[req.params.id] = fetchData(req.params.id); // grows forever
  res.json(cache[req.params.id]);
});
// FIX: Use LRU cache with max size
const LRU = require('lru-cache');
const cache = new LRU({ max: 500 }); // evicts oldest when full
```

1.5 libuv Thread Pool

Not all async operations use the event loop directly. Some use libuv's thread pool (default: 4 threads):

Thread pool operations: fs.readFile, crypto.pbkdf2, dns.lookup, zlib compression. These are CPU-heavy or use blocking OS APIs.

Event loop (epoll/kqueue): Network I/O (HTTP, TCP, UDP), fs.watch, timers. These use OS-level async APIs.

Critical setting: UV_THREADPOOL_SIZE=64 (increase from default 4 for I/O-heavy apps). NEXUS sets this to handle concurrent file logging + crypto operations.

```
// Increase thread pool BEFORE any async operations
```

```
process.env.UV_THREADPOOL_SIZE = 16;

// Verify with a benchmark
const crypto = require('crypto');
const start = Date.now();
for (let i = 0; i < 8; i++) {
  crypto.pbkdf2('password', 'salt', 100000, 512, 'sha512', () => {
    console.log(`Hash ${i}: ${Date.now() - start}ms`);
  });
}
// With UV_THREADPOOL_SIZE=4: first 4 finish ~same time, next 4 wait
// With UV_THREADPOOL_SIZE=8: all 8 finish ~same time
```

Chapter 1: Interview Questions

Interview Question	Answer
What is the event loop? Explain its phases.	The event loop is Node.js's mechanism for non-blocking I/O. It has 6 phases: Timers (setTimeout/setInterval), Pending Callbacks (deferred I/O), Idle/Prepare (internal), Poll (new I/O events), Check (setImmediate), Close Callbacks. Between each phase, microtasks (process.nextTick + Promises) are drained.
Difference between process.nextTick() and setImmediate()?	nextTick runs after current operation completes, BEFORE the event loop continues to the next phase. setImmediate runs in the Check phase of the next event loop iteration. nextTick has higher priority. Recursive nextTick can starve I/O; recursive setImmediate cannot.
Is Node.js single-threaded?	Yes and no. The JS execution is single-threaded (one call stack). But libuv provides a thread pool (default 4 threads) for file I/O, crypto, DNS, and compression. Network I/O uses OS-level async (epoll/kqueue), not threads. So Node.js is single-threaded for JS but multi-threaded for I/O.
What causes memory leaks in Node.js?	Top 3: (1) Closures holding references to large objects, (2) Event listeners added but never removed (especially in loops), (3) Global caches without eviction policies. Detect with: process.memoryUsage(), --inspect flag + Chrome DevTools heap snapshots, or clinic.js.
What is the V8 hidden class?	V8 creates hidden classes (internal type descriptors) for objects to enable fast property access. Objects with the same property names added in the same order share a hidden class. Adding properties dynamically or in different orders forces V8 to create new hidden classes, defeating inline caching and slowing property access by 10-100x.
How would you handle CPU-intensive tasks in Node.js?	Options: (1) Worker Threads — run CPU work in separate threads, (2) Child Processes — fork separate Node processes, (3) Cluster module — utilize all CPU cores, (4) Offload to a queue (Redis/RabbitMQ) and process in background workers, (5) Use native addons (C/C++) for extreme performance. For NEXUS, we use Worker Threads

	for AI inference and Cluster for multi-core request handling.
Explain backpressure in Node.js streams.	Backpressure occurs when a writable stream can't consume data as fast as a readable stream produces it. Without handling, data accumulates in memory causing OOM. Node's pipe() handles this automatically by pausing the readable when the writable's internal buffer exceeds highWaterMark. In manual implementations, check writable.write() return value — if false, pause reading until 'drain' event.
What is libuv and why does Node.js need it?	libuv is a C library that provides: (1) the event loop implementation, (2) async TCP/UDP sockets, (3) async DNS resolution, (4) file system operations, (5) thread pool for blocking operations, (6) child process management. Node needs it because V8 alone only executes JavaScript — it has no I/O capabilities. libuv bridges JS to the operating system.

BUILD TASK: NEXUS Foundation

Create the entry point for NEXUS: a bare Node.js HTTP server (no Express yet) that: (1) Listens on a configurable port from environment variables, (2) Logs the event loop utilization using perf_hooks.monitorEventLoopDelay(), (3) Has a /health endpoint returning process.memoryUsage() and uptime, (4) Gracefully handles SIGTERM/SIGINT for shutdown. HINTS: Use http.createServer(), process.env, and perf_hooks module.

Chapter 2: Modules, npm & Package Architecture

NEXUS Relevance: *NEXUS uses a plugin system that dynamically imports modules at runtime. Understanding CJS vs ESM and module resolution is essential.*

2.1 CommonJS vs ES Modules

Node.js supports two module systems. Understanding both is non-negotiable.

CommonJS (CJS)	ES Modules (ESM)
`require() — synchronous`	`import — asynchronous (can be top-level await)`
`module.exports / exports`	`export / export default`
`__dirname, __filename available`	`Use import.meta.url instead`
`Dynamic require: require(variable)`	`Dynamic import: await import(variable)`
`.js extension assumed`	`Must use .js extension explicitly`
`Circular deps return partial exports`	`Circular deps return live bindings`
`Default in Node.js`	`Need "type": "module" in package.json`

```
// === CommonJS ===
// math.js
function add(a, b) { return a + b; }
module.exports = { add };

// app.js
const { add } = require('./math');
console.log(add(2, 3)); // 5

// === ES Modules ===
// math.mjs (or .js with "type": "module")
export function add(a, b) { return a + b; }

// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3)); // 5

// === Dynamic Import (works in both) ===
async function loadPlugin(name) {
  const plugin = await import(`./plugins/${name}.js`);
  return plugin.default;  // NEXUS uses this for plugin loading!
}
```

2.2 Module Resolution Algorithm

When you write require('express'), Node.js searches in this exact order:

```
require('express')

1. Core modules? (fs, http, path) → YES → return core module
2. Starts with './' or '../' or '/'? → load as file/directory
3. Search node_modules:
   ./node_modules/express
   ../node_modules/express
   ../../node_modules/express
   ... (walk up to root)

For each directory, try:
```

```
   a. package.json → "main" field → load that file
   b. index.js
   c. index.node (native addon)

// This is why node_modules can be HUGE — each package
// can have its own nested node_modules with different versions
```

2.3 Package.json Deep Dive

```
{
  "name": "nexus-gateway",
  "version": "1.0.0",
  "type": "module",          // Enable ESM
  "main": "dist/index.js",   // CJS entry point
  "module": "dist/index.mjs", // ESM entry point
  "engines": {
    "node": ">=20.0.0"       // Enforce Node version
  },
  "scripts": {
    "start": "node dist/index.js",
    "dev": "nodemon --watch src src/index.js",
    "test": "jest --coverage",
    "lint": "eslint src/",
    "build": "tsc"
  },
  "dependencies": {           // Production deps
    "express": "^4.18.0",    // ^ = minor updates OK
    "pg": "~8.11.0"          // ~ = patch updates only
  },
  "devDependencies": {        // Dev-only deps
    "jest": "^29.0.0",
    "nodemon": "^3.0.0"
  },
  "peerDependencies": {       // Must be installed by consumer
    "react": ">=18.0.0"
  }
}
```

2.4 Monorepo & Workspace Architecture

Large projects like NEXUS use monorepos to organize multiple packages. npm workspaces (built-in since npm 7) make this easy:

```
nexus/
├── package.json          // root: { "workspaces": ["packages/*"] }
├── packages/
│   ├── gateway-core/     // Main gateway server
│   │   ├── package.json
│   │   └── src/
│   ├── auth-server/      // Authentication service
│   │   ├── package.json
│   │   └── src/
│   ├── plugin-engine/    // Plugin system
│   │   ├── package.json
│   │   └── src/
│   └── shared/           // Shared utilities
│       ├── package.json
│       └── src/
└── docker-compose.yml    // Orchestrate all services
```

Chapter 2: Interview Questions

Interview Question	Answer
CJS vs ESM — when to use which?	Use ESM for new projects (it's the standard, supports top-level await, tree-shaking). Use CJS when: working in existing CJS codebase, or need synchronous require() for conditional loading. NEXUS uses ESM throughout with dynamic import() for plugin loading.
What is the difference between dependencies and devDependencies?	dependencies are needed in production (express, pg, redis). devDependencies are only for development (jest, eslint, nodemon). When deploying, 'npm install --production' skips devDependencies, reducing image size. Getting this wrong bloats Docker images.
What is a circular dependency and how do you handle it?	When module A requires B and B requires A. In CJS, one module gets a partial (incomplete) export. In ESM, you get live bindings but may access undefined values if timing is wrong. Fix by: extracting shared code into a third module, or using dependency injection, or lazy loading with dynamic import().
Explain npm lock file. Why is package-lock.json important?	package-lock.json records the exact version of every installed package (including transitive deps). Without it, npm install might install different versions on different machines (due to ^ and ~ ranges). ALWAYS commit package-lock.json. Use 'npm ci' in CI/CD (installs from lock file exactly).

BUILD TASK: NEXUS Monorepo Setup

Initialize the NEXUS monorepo with npm workspaces. Create 4 packages: gateway-core, auth-server, plugin-engine, shared. Each should have its own package.json with appropriate dependencies. The shared package should export a logger utility and config loader that other packages import. HINTS: 'npm init -w packages/gateway-core', cross-package imports use the package name not relative paths.

Chapter 3: Asynchronous Mastery

NEXUS Relevance: *Every request in NEXUS triggers 3-5 async operations (auth check, rate limit lookup, upstream call, logging, metrics). Mastering async patterns prevents callback hell, memory leaks, and race conditions.*

3.1 The Evolution: Callbacks → Promises → async/await

Callback Pattern (Legacy but Still Tested)

```
// Node.js error-first callback convention
const fs = require('fs');

fs.readFile('/etc/passwd', 'utf8', (err, data) => {
  if (err) {
    console.error('Failed to read:', err.message);
    return;  // CRITICAL: always return after error
  }
  console.log(data);
});

// CALLBACK HELL (why we moved away)
getUser(id, (err, user) => {
  if (err) return handleError(err);
  getOrders(user.id, (err, orders) => {
    if (err) return handleError(err);
    getPayments(orders[0].id, (err, payments) => {
      if (err) return handleError(err);
      // 3 levels deep... and it gets worse
    });
  });
});
```

Promises (The Foundation of Modern Async)

```
// Creating a Promise
function readFileAsync(path) {
  return new Promise((resolve, reject) => {
    fs.readFile(path, 'utf8', (err, data) => {
      if (err) reject(err);
      else resolve(data);
    });
  });
}

// Chaining (flat, no nesting!)
readFileAsync('/config.json')
  .then(data => JSON.parse(data))
  .then(config => connectDB(config.db))
  .then(db => db.query('SELECT * FROM users'))
  .catch(err => console.error(err))  // catches ANY error above
  .finally(() => console.log('Done'));

// Promise.all — parallel execution (NEXUS uses this everywhere)
const [user, orders, settings] = await Promise.all([
  fetchUser(id),
  fetchOrders(id),
  fetchSettings(id),
]);
// All 3 run in PARALLEL, total time = max(individual times)
// If ANY fails, ALL fail — use Promise.allSettled for fault tolerance
```

```
// Promise.allSettled — parallel with individual error handling
const results = await Promise.allSettled([
  fetchUser(id),       // might fail
  fetchOrders(id),     // might fail
  fetchSettings(id),   // might fail
]);
results.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.error(r.reason);
});
```

async/await (The Standard)

```
// Clean, readable, debuggable
async function processRequest(req) {
  try {
    const user = await authenticate(req.token);
    const allowed = await checkRateLimit(user.id);
    if (!allowed) throw new AppError('Rate limited', 429);

    const [upstream, cache] = await Promise.all([
      forwardToUpstream(req),
      checkCache(req.url),
    ]);

    return cache ?? upstream;  // nullish coalescing
  } catch (error) {
    if (error instanceof AppError) {
      return { status: error.code, body: error.message };
    }
    // Unexpected errors — log and rethrow
    logger.error('Unhandled:', error);
    throw error;
  }
}

// COMMON MISTAKE: Forgetting await
async function bad() {
  const promise = fetchData();  // NOT awaited!
  console.log(promise);         // Promise { <pending> }
  // If this throws, error is UNHANDLED
}

// COMMON MISTAKE: Sequential when parallel is possible
async function slow() {
  const a = await fetch('/api/a');  // waits 200ms
  const b = await fetch('/api/b');  // waits 200ms AFTER a
  // Total: 400ms
}
async function fast() {
  const [a, b] = await Promise.all([
    fetch('/api/a'),  // starts immediately
    fetch('/api/b'),  // starts immediately
  ]);
  // Total: 200ms (parallel!)
}
```

3.2 Streams — The Secret Weapon

Streams process data in chunks instead of loading everything into memory. Critical for handling large files, video, and high-throughput systems like NEXUS.

```
const { createReadStream, createWriteStream } = require('fs');
const { Transform, pipeline } = require('stream');
const { promisify } = require('util');
const pipelineAsync = promisify(pipeline);

// Transform stream: modify data as it flows through
class RequestLogger extends Transform {
  _transform(chunk, encoding, callback) {
    const log = JSON.parse(chunk.toString());
    log.timestamp = new Date().toISOString();
    log.processed = true;
    this.push(JSON.stringify(log) + '\n');
    callback();  // signal we're ready for more data
  }
}

// Pipeline with error handling (ALWAYS use pipeline, not .pipe())
await pipelineAsync(
  createReadStream('access.log'),
  new RequestLogger(),
  createWriteStream('processed.log')
);
// Processes a 10GB log file using only ~64KB of memory!

// NEXUS uses streams for:
// - Streaming request/response bodies to upstream services
// - Processing large webhook payloads
// - Real-time log processing pipeline
```

3.3 Error Handling Patterns (Production-Grade)

```
// Custom error classes (NEXUS error hierarchy)
class AppError extends Error {
  constructor(message, statusCode, isOperational = true) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = isOperational; // vs programming errors
    Error.captureStackTrace(this, this.constructor);
  }
}
class NotFoundError extends AppError {
  constructor(resource) {
    super(`${resource} not found`, 404);
  }
}
class RateLimitError extends AppError {
  constructor(retryAfter) {
    super('Rate limit exceeded', 429);
    this.retryAfter = retryAfter;
  }
}

// Global unhandled error catchers (MUST have in production)
process.on('unhandledRejection', (reason, promise) => {
  logger.error('Unhandled Rejection:', reason);
  // In production: graceful shutdown
  process.exit(1);
});
process.on('uncaughtException', (error) => {
  logger.error('Uncaught Exception:', error);
  // ALWAYS exit — state may be corrupted
  process.exit(1);
});
```

Chapter 3: Interview Questions

Interview Question	Answer
Promise.all vs Promise.allSettled vs Promise.race vs Promise.any?	all: resolves when ALL resolve, rejects on FIRST rejection. allSettled: waits for ALL (never rejects), returns [{status, value/reason}]. race: resolves/rejects with the FIRST promise to settle. any: resolves with FIRST fulfillment, rejects only if ALL reject (AggregateError). Use all for parallel tasks that must all succeed, allSettled for independent tasks, race for timeouts, any for fallback strategies.
What are streams? Name the 4 types.	Streams are collections of data that might not be available all at once. 4 types: Readable (source: fs.createReadStream), Writable (destination: fs.createWriteStream), Duplex (both: net.Socket), Transform (modify data passing through: zlib.createGzip). Always use pipeline() instead of .pipe() for proper error handling and cleanup.
How do you handle errors in async/await?	try/catch blocks for known error paths. Custom error classes (AppError) with isOperational flag to distinguish business errors from programmer errors. Global handlers: process.on('unhandledRejection') and process.on('uncaughtException'). Express: pass errors to next(error) and use a centralized error-handling middleware. NEVER silently swallow errors.
What is the difference between concurrency and parallelism in Node.js?	Concurrency: handling multiple tasks by switching between them (event loop does this — single thread). Parallelism: executing tasks simultaneously on multiple CPU cores (requires Worker Threads or Cluster). Node.js is concurrent by default (event loop) but not parallel. Use Worker Threads for CPU parallelism.
Explain async iterators and for-await-of.	Async iterators produce values asynchronously (Symbol.asyncIterator). for-await-of consumes them: 'for await (const chunk of readableStream) { ... }'. Use cases: reading streams, paginating API results, processing database cursors. Available since Node 10. Replaces manual stream event handling with clean syntax.

BUILD TASK: NEXUS Request Pipeline

Build a request processing pipeline using async/await and streams. It should: (1) Accept an incoming HTTP request, (2) Run it through 3 async middleware functions in sequence (auth, rateLimit, transform), (3) Forward the request body to an upstream URL using streams (not buffering the entire body), (4) Return the upstream response, streamed back to the client. Use Promise.all for parallel checks (auth + rateLimit). HINTS: http.request() for upstream, stream.pipeline() for body forwarding, custom AppError for failures.

Chapter 4: HTTP Deep Dive — Building a Server From Scratch

NEXUS Relevance: *NEXUS IS an HTTP server at its core. Understanding the raw http module, request lifecycle, headers, status codes, and keep-alive is essential. We build the foundation before adding Express.*

4.1 The HTTP Request Lifecycle

```
// Raw HTTP server — no frameworks
const http = require('http');
const { URL } = require('url');

const server = http.createServer((req, res) => {
  // 1. Parse the URL
  const url = new URL(req.url, `http://${req.headers.host}`);
  const path = url.pathname;
  const query = Object.fromEntries(url.searchParams);

  // 2. Parse the body (for POST/PUT/PATCH)
  let body = '';
  req.on('data', chunk => { body += chunk; });
  req.on('end', () => {
    const parsed = body ? JSON.parse(body) : {};

    // 3. Route the request
    if (req.method === 'GET' && path === '/api/health') {
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ status: 'ok', uptime: process.uptime() }));
    }
    else if (req.method === 'POST' && path === '/api/routes') {
      // Register a new route in NEXUS
      res.writeHead(201, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ created: true, route: parsed }));
    }
    else {
      res.writeHead(404, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ error: 'Not Found' }));
    }
  });
});

server.listen(3000, () => {
  console.log('NEXUS raw server running on :3000');
});

// Connection management
server.keepAliveTimeout = 65000; // > ALB's 60s default
server.headersTimeout = 66000;   // > keepAliveTimeout
server.maxHeadersCount = 100;
server.timeout = 120000;         // 2 min request timeout
```

4.2 HTTP Headers Every Backend Engineer Must Know

Header	Purpose	Example
Content-Type	What format the body is in	`application/json; charset=utf-8`
Authorization	Authentication credentials	`Bearer eyJhbGciOiJIUzI1...`
Cache-Control	Caching directives	`public, max-age=3600, s-maxage=86400`
ETag	Resource version identifier	`"33a64df551425fcc55e4"`

X-Request-ID	Trace ID for distributed tracing	`550e8400-e29b-41d4-a716-446655440000`
X-RateLimit-Remaining	Remaining requests in window	`97`
Content-Encoding	Compression algorithm	`gzip`
Retry-After	When to retry (429/503)	`120 (seconds)`
Strict-Transport-Security	Force HTTPS	`max-age=31536000; includeSubDomains`
X-Content-Type-Options	Prevent MIME sniffing	`nosniff`

4.3 HTTP Status Codes — The Complete Mental Model

```
// 2xx — Success
200  // OK — GET/PUT/PATCH succeeded
201  // Created — POST created a resource
204  // No Content — DELETE succeeded (no body)

// 3xx — Redirection
301  // Moved Permanently — URL changed forever (SEO)
302  // Found — Temporary redirect
304  // Not Modified — use cached version (ETag match)

// 4xx — Client Errors
400  // Bad Request — malformed input
401  // Unauthorized — not authenticated (no/bad token)
403  // Forbidden — authenticated but not authorized
404  // Not Found — resource doesn't exist
409  // Conflict — duplicate resource / version conflict
422  // Unprocessable Entity — valid JSON but bad data
429  // Too Many Requests — rate limited

// 5xx — Server Errors
500  // Internal Server Error — unexpected bug
502  // Bad Gateway — upstream server error (NEXUS uses this!)
503  // Service Unavailable — server overloaded / maintenance
504  // Gateway Timeout — upstream didn't respond in time
```

4.4 System Design: Connection Pooling & Keep-Alive

When NEXUS forwards requests to upstream services, creating a new TCP connection for each request is wasteful. Connection pooling reuses existing connections:

```
const http = require('http');

// Create a persistent agent with connection pooling
const agent = new http.Agent({
  keepAlive: true,          // reuse connections
  maxSockets: 100,          // max concurrent per host
  maxTotalSockets: 1000,    // max concurrent total
  maxFreeSockets: 50,       // max idle connections to keep
  timeout: 60000,           // socket timeout
  scheduling: 'lifo',       // reuse most recently used (warmer)
});

// Use the agent for upstream requests
function forwardRequest(upstreamUrl, req) {
  return new Promise((resolve, reject) => {
    const upstream = http.request(upstreamUrl, { agent }, resolve);
    upstream.on('error', reject);
    req.pipe(upstream);  // stream body directly
  });
```

```
}

// SYSTEM DESIGN INTERVIEW:
// Q: How would you handle 10,000 concurrent requests?
// A: Connection pooling (reuse TCP connections to upstreams),
//    Keep-alive (avoid 3-way handshake overhead),
//    Load balancing across multiple NEXUS instances,
//    Circuit breaker for failing upstreams
```

Chapter 4: Interview Questions

Interview Question	Answer
What happens when you type a URL in the browser?	DNS resolution → TCP 3-way handshake → TLS handshake (HTTPS) → HTTP request sent → Server processes → HTTP response → Browser renders HTML → Loads CSS/JS/images (more requests). Each step is an interview deep-dive topic.
GET vs POST vs PUT vs PATCH vs DELETE?	GET: read resource (idempotent, cacheable). POST: create resource (not idempotent). PUT: replace entire resource (idempotent). PATCH: partial update (not idempotent). DELETE: remove resource (idempotent). Idempotent = same request produces same result.
What is HTTP/2 and why does it matter?	HTTP/2 adds: multiplexing (multiple requests on one TCP connection), header compression (HPACK), server push, stream prioritization. Result: much faster than HTTP/1.1 for many concurrent requests. Node.js supports HTTP/2 via the http2 module.
Explain CORS. How do you handle it?	Cross-Origin Resource Sharing. Browsers block requests from one origin to another for security. Server must respond with Access-Control-Allow-Origin header. Preflight requests (OPTIONS) check if the actual request is allowed. Use cors() middleware in Express or set headers manually.
What is the difference between 401 and 403?	401 Unauthorized = you haven't proven who you are (missing/invalid auth token). 403 Forbidden = you've proven who you are, but you don't have permission. A common mistake is returning 403 when 401 is correct.

BUILD TASK: NEXUS Router

Build a custom HTTP router (no Express) for NEXUS with: (1) Route registration: router.get('/api/:service/*', handler), (2) Path parameter extraction (:service → req.params.service), (3) Wildcard matching (/* catches everything after), (4) Request body parsing as a stream (don't buffer large bodies), (5) Connection pooling agent for upstream forwarding. HINTS: Use URL patterns to match routes, RegExp for path params, http.Agent for pooling.

Chapter 5: Express.js — Middleware Architecture

NEXUS Relevance: *NEXUS uses Express as its HTTP framework layer. The middleware pattern is the backbone of the entire gateway — every request passes through a chain of middleware functions.*

5.1 The Middleware Pattern (Express's Core Innovation)

Every Express middleware is a function with signature (req, res, next). The request flows through middleware in order. Each middleware can: modify req/res, send a response (ending the chain), or call next() to pass to the next middleware.

```
const express = require('express');
const app = express();

// Middleware execution order matters!

// 1. Request timing (runs for ALL requests)
app.use((req, res, next) => {
  req.startTime = process.hrtime.bigint();
  res.on('finish', () => {
    const duration = Number(process.hrtime.bigint()-req.startTime)/1e6;
    console.log(`${req.method} ${req.url} ${res.statusCode} ${duration}ms`);
  });
  next();
});

// 2. Body parsing
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));

// 3. Security headers
app.use((req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('X-XSS-Protection', '0');
  next();
});

// 4. Routes
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok' });
});

// 5. 404 handler (after all routes)
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});

// 6. Error handler (4 params — Express signature)
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  res.status(status).json({
    error: err.message,
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
  });
});
```

5.2 Advanced Middleware Patterns

Factory Middleware (Configurable)

```
// Middleware factory — returns middleware configured with options
function rateLimit({ windowMs = 60000, max = 100 } = {}) {
  const hits = new Map();

  // Cleanup old entries periodically
  setInterval(() => {
    const now = Date.now();
    for (const [key, data] of hits) {
      if (now - data.resetTime > windowMs) hits.delete(key);
    }
  }, windowMs);

  return (req, res, next) => {
    const key = req.ip;
    const now = Date.now();
    const record = hits.get(key) || { count: 0, resetTime: now };

    if (now - record.resetTime > windowMs) {
      record.count = 0;
      record.resetTime = now;
    }

    record.count++;
    hits.set(key, record);

    res.setHeader('X-RateLimit-Limit', max);
    res.setHeader('X-RateLimit-Remaining', Math.max(0, max-record.count));

    if (record.count > max) {
      res.setHeader('Retry-After', Math.ceil(windowMs / 1000));
      return res.status(429).json({ error: 'Too many requests' });
    }
    next();
  };
}

// Usage
app.use('/api/', rateLimit({ windowMs: 60000, max: 100 }));
app.use('/api/auth', rateLimit({ windowMs: 60000, max: 10 })); // stricter
```

Async Middleware Wrapper

```
// Express doesn't catch async errors by default!
// This wrapper fixes that
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// Without wrapper: unhandled rejection (Express can't catch)
app.get('/api/users', async (req, res) => {
  const users = await db.query('SELECT * FROM users');  // if this throws?
  res.json(users);  // Error goes to unhandledRejection, not error handler
});

// With wrapper: error properly forwarded to error handler
app.get('/api/users', asyncHandler(async (req, res) => {
  const users = await db.query('SELECT * FROM users');
  res.json(users);  // If this throws, caught and sent to error middleware
}));

// NEXUS wraps ALL route handlers with asyncHandler
```

5.3 Request Validation with Zod (Production Standard)

```
const { z } = require('zod');

// Define schema
const CreateRouteSchema = z.object({
  path: z.string().min(1).startsWith('/'),
  upstream: z.string().url(),
  method: z.enum(['GET','POST','PUT','PATCH','DELETE']),
  timeout: z.number().min(100).max(30000).default(5000),
  retries: z.number().min(0).max(5).default(3),
  plugins: z.array(z.string()).optional(),
});

// Validation middleware factory
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(422).json({
        error: 'Validation failed',
        details: result.error.issues.map(i => ({
          field: i.path.join('.'),
          message: i.message,
        })),
      });
    }
    req.validated = result.data;  // attach cleaned data
    next();
  };
}

app.post('/api/routes', validate(CreateRouteSchema),
  asyncHandler(async (req, res) => {
    const route = await RouteService.create(req.validated);
    res.status(201).json(route);
  })
);
```

5.4 Express App Architecture (Production Pattern)

```
// Project structure for NEXUS
nexus-gateway/
├── src/
│   ├── index.js            // Entry point
│   ├── app.js              // Express app setup
│   ├── config/
│   │   ├── index.js        // Config loader (env vars)
│   │   └── database.js     // DB connection config
│   ├── middleware/
│   │   ├── auth.js         // JWT verification
│   │   ├── rateLimit.js    // Rate limiting
│   │   ├── validate.js     // Zod validation
│   │   ├── errorHandler.js // Centralized error handling
│   │   └── requestId.js    // Add X-Request-ID
│   ├── routes/
│   │   ├── index.js        // Route aggregator
│   │   ├── health.js       // /health endpoints
│   │   ├── routes.js       // /api/routes CRUD
│   │   └── proxy.js        // /* catch-all proxy
│   ├── services/
│   │   ├── routeService.js // Business logic
│   │   ├── proxyService.js // Upstream forwarding
│   │   └── authService.js  // Auth logic
│   ├── models/             // Database models
```

```
│   ├── utils/
│   │   ├── logger.js       // Winston logger
│   │   ├── errors.js       // Custom error classes
│   │   └── helpers.js
│   └── plugins/            // Plugin directory
├── tests/
├── docker-compose.yml
├── Dockerfile
└── package.json
```

Chapter 5: Interview Questions

Interview Question	Answer
How does Express middleware work internally?	Express maintains an array of middleware/route handlers. On each request, it iterates through the array. Each handler receives (req, res, next). Calling next() moves to the next handler. Not calling next() or sending a response ends the chain. Error handlers have 4 params (err, req, res, next). Order of app.use() determines execution order.
How do you structure a large Express application?	Separation of concerns: routes/ (HTTP layer, validation), services/ (business logic, no Express dependency), models/ (database), middleware/ (cross-cutting concerns), config/ (environment), utils/ (helpers, errors). Each route file handles one resource. Services are testable without HTTP. This is the Controller-Service-Repository pattern.
How do you handle errors in Express?	Synchronous: throw in route handler. Async: wrap in asyncHandler that catches and calls next(error). Centralized error middleware (4 params) at the end of middleware chain. Distinguish operational errors (AppError, expected) from programmer errors (TypeError, unexpected). Log all, only expose operational errors to client.
Express vs Koa vs Fastify — when to use which?	Express: mature, huge ecosystem, most popular (80%+ market). Koa: lightweight, modern (async/await native), no built-in middleware. Fastify: fastest (2-3x Express), schema-based validation, TypeScript-friendly. For NEXUS: Express for ecosystem + hiring advantage. For performance-critical new projects: Fastify.
How do you prevent common Express security vulnerabilities?	1) helmet() for security headers, 2) express.json({ limit: '10mb' }) to prevent large payload DoS, 3) express-rate-limit for rate limiting, 4) cors() with specific origins (not *), 5) Input validation with Zod/Joi, 6) Parameterized queries (never string concat SQL), 7) httpOnly + secure cookies.

BUILD TASK: NEXUS Express Foundation

Restructure the raw HTTP server into Express with the production architecture above. Implement: (1) The full middleware chain (timing, body parsing, security headers, request ID), (2) Factory rate limiter middleware with configurable windows, (3) Zod validation for a POST /api/routes endpoint, (4) asyncHandler wrapper for all routes, (5) Centralized error handler that returns clean JSON errors. HINTS: Use the folder structure shown above. Separate app.js (Express setup) from index.js (server start + graceful shutdown).

Chapter 6: Authentication & Authorization

NEXUS Relevance: *NEXUS's Auth Server handles JWT issuance, API key management, OAuth2 flows, and RBAC for multi-tenant access control.*

6.1 JWT (JSON Web Tokens) — Deep Dive

```
// JWT Structure: header.payload.signature
// Header: { "alg": "HS256", "typ": "JWT" }
// Payload: { "sub": "user123", "role": "admin", "iat": 1234 }
// Signature: HMACSHA256(base64(header)+'.'+base64(payload), secret)

const jwt = require('jsonwebtoken');

// ---- TOKEN GENERATION ----
function generateTokens(user) {
  const accessToken = jwt.sign(
    { sub: user.id, role: user.role, email: user.email },
    process.env.JWT_SECRET,
    { expiresIn: '15m', issuer: 'nexus-auth' }
  );

  const refreshToken = jwt.sign(
    { sub: user.id, type: 'refresh' },
    process.env.JWT_REFRESH_SECRET,
    { expiresIn: '7d', issuer: 'nexus-auth' }
  );

  return { accessToken, refreshToken };
}

// ---- TOKEN VERIFICATION MIDDLEWARE ----
function authenticate(req, res, next) {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer ')) {
    throw new AppError('No token provided', 401);
  }
  try {
    const token = header.slice(7);
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      throw new AppError('Token expired', 401);
    }
    throw new AppError('Invalid token', 401);
  }
}

// ---- RBAC MIDDLEWARE ----
function authorize(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      throw new AppError('Insufficient permissions', 403);
    }
    next();
  };
}

// Usage
app.delete('/api/routes/:id',
  authenticate,
```

```
  authorize('admin', 'owner'),
  asyncHandler(deleteRoute)
);
```

6.2 Password Hashing (bcrypt)

```
const bcrypt = require('bcrypt');

// NEVER store plain passwords. NEVER use MD5/SHA256 for passwords.
// bcrypt is slow BY DESIGN (prevents brute force)

async function register(email, password) {
  // Salt rounds = 12 is the sweet spot (2024)
  // Higher = slower hashing but more secure
  const hashedPassword = await bcrypt.hash(password, 12);
  return db.query(
    'INSERT INTO users (email, password) VALUES ($1, $2) RETURNING id',
    [email, hashedPassword]
  );
}

async function login(email, password) {
  const user = await db.query(
    'SELECT * FROM users WHERE email = $1', [email]
  );
  if (!user) throw new AppError('Invalid credentials', 401);

  const valid = await bcrypt.compare(password, user.password);
  if (!valid) throw new AppError('Invalid credentials', 401);
  // Don't say 'wrong password' — reveals that email exists

  return generateTokens(user);
}
```

6.3 API Key Authentication (For Machine-to-Machine)

```
const crypto = require('crypto');

// Generate API key (prefix + random bytes)
function generateApiKey() {
  const prefix = 'nxs_';  // NEXUS prefix for easy identification
  const key = crypto.randomBytes(32).toString('hex');
  const fullKey = prefix + key;

  // Store HASH of key, not the key itself (like passwords)
  const hashedKey = crypto
    .createHash('sha256')
    .update(fullKey)
    .digest('hex');

  return { fullKey, hashedKey };  // Show fullKey to user ONCE
}

// Verify API key middleware
async function authenticateApiKey(req, res, next) {
  const key = req.headers['x-api-key'];
  if (!key) return next(); // fall through to JWT auth

  const hashedKey = crypto.createHash('sha256').update(key).digest('hex');
  const apiKey = await db.query(
    'SELECT * FROM api_keys WHERE key_hash = $1 AND revoked = false',
    [hashedKey]
  );
```

```
  if (!apiKey) throw new AppError('Invalid API key', 401);
  req.user = { id: apiKey.user_id, role: apiKey.role };
  next();
}
```

6.4 OAuth2 Flow (System Design)

OAuth2 is used when users log in via Google/GitHub. NEXUS supports this for developer authentication:

```
// Simplified OAuth2 Authorization Code Flow
// 1. User clicks 'Login with GitHub'
// 2. Redirect to: github.com/login/oauth/authorize?
//    client_id=NEXUS_ID&redirect_uri=nexus.com/callback&scope=user:email
// 3. User approves, GitHub redirects back with ?code=XXXXX
// 4. NEXUS exchanges code for access_token (server-to-server)
// 5. NEXUS uses access_token to fetch user profile from GitHub API
// 6. Create/update user in DB, issue NEXUS JWT

app.get('/auth/github/callback', asyncHandler(async (req, res) => {
  const { code } = req.query;

  // Exchange code for token
  const tokenRes = await fetch('https://github.com/login/oauth/access_token', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', Accept: 'application/json' },
    body: JSON.stringify({
      client_id: process.env.GITHUB_CLIENT_ID,
      client_secret: process.env.GITHUB_CLIENT_SECRET,
      code,
    }),
  });
  const { access_token } = await tokenRes.json();

  // Fetch user profile
  const userRes = await fetch('https://api.github.com/user', {
    headers: { Authorization: `Bearer ${access_token}` },
  });
  const profile = await userRes.json();

  // Upsert user in our DB
  const user = await UserService.upsert({
    githubId: profile.id,
    email: profile.email,
    name: profile.name,
  });

  const tokens = generateTokens(user);
  res.redirect(`/dashboard?token=${tokens.accessToken}`);
}));
```

Chapter 6: Interview Questions

Interview Question	Answer
JWT vs Session-based auth — pros/cons?	JWT: stateless (no server storage), works across services (microservices), scalable. But can't be revoked until expiry (fix: short expiry + refresh tokens or blacklist in Redis). Sessions: server stores state, easy revocation, but requires sticky sessions or shared session store (Redis). Use JWT for APIs, sessions for traditional web apps.

How do you handle token refresh securely?	Access token: short-lived (15min). Refresh token: long-lived (7d), stored in httpOnly cookie (not localStorage — XSS vulnerable). On access token expiry, client sends refresh token to /auth/refresh. Server verifies refresh token, issues new access+refresh pair. Rotate refresh tokens on each use (prevents reuse if stolen).
What is RBAC? How do you implement it?	Role-Based Access Control. Users have roles (admin, editor, viewer). Roles have permissions (create:route, delete:route, read:metrics). Check: user.role → role.permissions → includes required permission. Store in DB: users, roles, permissions, role_permissions (many-to-many). Middleware checks user's role against required roles for each endpoint.
Why bcrypt over SHA256 for passwords?	SHA256 is fast (billions/sec on GPU). bcrypt is intentionally slow (configurable cost factor). Attacker can crack SHA256 passwords with dictionary attacks in minutes. bcrypt with cost factor 12 takes ~250ms per hash — making brute force impractical. Also: bcrypt includes a salt automatically, preventing rainbow table attacks.
How do you secure API keys?	Never store plain keys in DB — hash them (SHA256 is fine for API keys, unlike passwords, because keys are random not guessable). Transmit via HTTPS only. Include a prefix (nxs_) for easy identification in logs. Support revocation. Scope keys to specific permissions. Rate limit per key. Log all key usage for audit trails.

BUILD TASK: NEXUS Auth Server

Build the complete auth subsystem: (1) User registration with bcrypt password hashing, (2) Login returning JWT access + refresh tokens, (3) Token refresh endpoint with refresh token rotation, (4) API key generation + verification, (5) RBAC middleware with admin/developer/viewer roles, (6) Auth middleware that supports BOTH JWT (Authorization: Bearer) and API keys (X-API-Key). HINTS: Use two JWT secrets (access vs refresh). Store refresh tokens in Redis with TTL. Hash API keys with SHA256.

Chapter 7: Database Engineering — PostgreSQL, MongoDB, Redis

NEXUS Relevance: *NEXUS uses PostgreSQL for route configs + user data, Redis for rate limiting + caching + pub/sub.*

7.1 PostgreSQL Schema for NEXUS

```
-- NEXUS Database Schema
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) UNIQUE NOT NULL,
  password    VARCHAR(255) NOT NULL,
  role        VARCHAR(50) DEFAULT 'developer',
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE routes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  path        VARCHAR(500) NOT NULL,
  upstream    VARCHAR(500) NOT NULL,
  method      VARCHAR(10) DEFAULT 'GET',
  timeout_ms  INTEGER DEFAULT 5000,
  retries     INTEGER DEFAULT 3,
  plugins     JSONB DEFAULT '[]',
  active      BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, path, method)
);

CREATE TABLE request_logs (
  id          BIGSERIAL PRIMARY KEY,
  route_id    UUID REFERENCES routes(id),
  status_code INTEGER,
  latency_ms  INTEGER,
  ip          INET,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- INDEXES
CREATE INDEX idx_routes_user ON routes(user_id, path);
CREATE INDEX idx_logs_created ON request_logs(created_at DESC);
CREATE INDEX idx_logs_route ON request_logs(route_id, created_at DESC);
```

Advanced Queries: Window Functions & CTEs

```
-- Top 5 routes per user by request count (window function)
SELECT * FROM (
  SELECT r.path, r.user_id, COUNT(l.id) AS reqs,
    AVG(l.latency_ms) AS avg_latency,
    ROW_NUMBER() OVER (PARTITION BY r.user_id ORDER BY COUNT(l.id) DESC) AS rank
  FROM routes r LEFT JOIN request_logs l ON l.route_id = r.id
  WHERE l.created_at > NOW() - INTERVAL '24 hours'
  GROUP BY r.id
) ranked WHERE rank <= 5;

-- Error rate by hour (CTE)
WITH hourly AS (
  SELECT date_trunc('hour', created_at) AS hour,
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE status_code >= 500) AS errors
  FROM request_logs WHERE created_at > NOW() - INTERVAL '24 hours'
```

```
  GROUP BY 1
)
SELECT hour, total, errors,
  ROUND(errors::numeric / NULLIF(total,0) * 100, 2) AS error_rate
FROM hourly ORDER BY hour DESC;
```

Transactions & Connection Pooling

```
const { Pool } = require('pg');
const pool = new Pool({ max: 20, idleTimeoutMillis: 30000 });

// Transaction: atomic route creation + audit log
async function createRoute(userId, data) {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const route = await client.query(
      `INSERT INTO routes (user_id, path, upstream, method)
       VALUES ($1, $2, $3, $4) RETURNING *`,
      [userId, data.path, data.upstream, data.method]
    );
    await client.query(
      `INSERT INTO audit_log (user_id, action, resource_id)
       VALUES ($1, 'CREATE_ROUTE', $2)`,
      [userId, route.rows[0].id]
    );
    await client.query('COMMIT');
    return route.rows[0];
  } catch (e) {
    await client.query('ROLLBACK');
    throw e;
  } finally {
    client.release(); // ALWAYS release!
  }
}
```

7.2 Redis — Rate Limiting, Caching, Pub/Sub

```
const Redis = require('ioredis');
const redis = new Redis(process.env.REDIS_URL);

// SLIDING WINDOW RATE LIMITER (sorted sets)
async function checkRateLimit(userId, limit = 100, windowSec = 60) {
  const key = `ratelimit:${userId}`;
  const now = Date.now();
  const pipe = redis.pipeline();
  pipe.zremrangebyscore(key, 0, now - windowSec * 1000);
  pipe.zadd(key, now, `${now}:${Math.random()}`);
  pipe.zcard(key);
  pipe.expire(key, windowSec);
  const results = await pipe.exec();
  const count = results[2][1];
  return { allowed: count <= limit, remaining: Math.max(0, limit-count) };
}

// CACHING with TTL
async function getCachedRoute(path) {
  const cached = await redis.get(`route:${path}`);
  if (cached) return JSON.parse(cached);
  const route = await db.query('SELECT * FROM routes WHERE path=$1', [path]);
  if (route.rows[0]) {
    await redis.setex(`route:${path}`, 300, JSON.stringify(route.rows[0]));
  }
```

```
  return route.rows[0];
}

// PUB/SUB for cache invalidation across instances
const sub = new Redis();
sub.subscribe('nexus:config:update');
sub.on('message', (ch, msg) => {
  const { path } = JSON.parse(msg);
  localCache.del(`route:${path}`); // invalidate in-memory cache
});
```

7.3 System Design: CAP Theorem

CAP: Consistency, Availability, Partition Tolerance — pick 2. Since partitions always happen, you choose CP (PostgreSQL — consistent but may be unavailable) or AP (Redis/MongoDB — available but eventually consistent). NEXUS: PostgreSQL for source of truth (CP), Redis for caching (AP).

Chapter 7: Interview Questions

Interview Question	Answer
SQL vs NoSQL — when?	SQL: structured data, relationships, ACID, complex queries (joins, window functions). NoSQL: flexible schemas, horizontal scaling, denormalized data. Redis: caching, sessions, rate limiting. Most apps: SQL + Redis. Add MongoDB only if flexible schemas essential.
Explain indexing. When does it hurt?	B-tree enabling O(log n) lookups. Hurts: write amplification (every INSERT/UPDATE/DELETE updates index), memory usage. Composite indexes: leftmost prefix rule. Always verify with EXPLAIN ANALYZE.
Connection pooling — why?	New connection = 20-50ms TCP+auth overhead. 1000 concurrent requests = 1000 connections killing DB. Pool: 10-20 persistent connections, queue when busy. Always set max pool size.
ACID vs BASE?	ACID (SQL): Atomicity, Consistency, Isolation, Durability — strong guarantees. BASE (NoSQL): Basically Available, Soft state, Eventually consistent — high throughput. ACID for financial data, BASE for analytics/logs.
Sharding strategies?	Hash-based (userId%N), range-based (A-M/N-Z), geography. Need when: single DB can't handle writes, dataset too large. Downsides: no cross-shard joins, rebalancing painful. Delay sharding — optimize with indexes, replicas, caching first.

BUILD TASK: NEXUS Data Layer

Build: (1) PostgreSQL schema with all tables + indexes, (2) Connection pool with pg-pool, (3) Transactions for route creation + audit, (4) Redis sliding-window rate limiter, (5) Redis caching with TTL, (6) Redis Pub/Sub for config propagation.

Chapter 8: ORM — Prisma & Query Optimization

NEXUS Relevance: *Prisma provides type-safe database access, auto-migrations, and solves the N+1 problem.*

8.1 Prisma Schema & CRUD

```
// prisma/schema.prisma
model User {
  id       String  @id @default(uuid())
  email    String  @unique
  password String
  role     Role    @default(DEVELOPER)
  routes   Route[]
  @@map("users")
}
model Route {
  id       String @id @default(uuid())
  userId   String @map("user_id")
  path     String
  upstream String
  method   String @default("GET")
  plugins  Json   @default("[]")
  active   Boolean @default(true)
  user     User @relation(fields:[userId],references:[id])
  logs     RequestLog[]
  @@unique([userId,path,method])
  @@map("routes")
}
enum Role { ADMIN DEVELOPER VIEWER }

// Cursor-based pagination
async function listRoutes(userId, cursor, limit=20) {
  const routes = await prisma.route.findMany({
    where: { userId, active: true },
    take: limit + 1,
    ...(cursor && { cursor: { id: cursor }, skip: 1 }),
    orderBy: { createdAt: 'desc' },
  });
  const hasMore = routes.length > limit;
  return { data: hasMore ? routes.slice(0,-1) : routes,
    nextCursor: hasMore ? routes[limit-1].id : null, hasMore };
}

// N+1 FIX: use include (generates JOIN)
const users = await prisma.user.findMany({
  include: { routes: true }, // single JOIN query
});
```

Chapter 8: Interview Questions

Interview Question	Answer
N+1 problem?	Fetch N items, then 1 query each for related data = N+1 queries. Fix: include/populate (ORM JOIN), DataLoader (batch+cache), or manual JOIN.
ORM vs raw SQL?	ORM: type-safe CRUD, migrations, 80% of queries. Raw SQL: window functions, CTEs, bulk ops, performance-critical 20%. Prisma supports $queryRaw.

BUILD TASK: NEXUS Prisma Layer
(1) Full Prisma schema, (2) Cursor pagination, (3) Raw SQL analytics: top 10 routes by request count with window functions.

Chapter 9: API Design — REST & GraphQL

NEXUS Relevance: *NEXUS management API uses REST with proper versioning, pagination, and idempotency.*

Code & Implementation

```
// REST Best Practices
GET    /api/v1/routes          // list
POST   /api/v1/routes          // create
GET    /api/v1/routes/:id      // get one
PATCH  /api/v1/routes/:id      // update
DELETE /api/v1/routes/:id      // delete

// Consistent response envelope
{ status:'success', data:{...}, meta:{ page:{cursor,hasMore} } }

// Idempotency (prevent duplicate POST)
app.post('/api/v1/routes', async (req, res) => {
  const key = req.headers['idempotency-key'];
  if (key) {
    const cached = await redis.get(`idem:${key}`);
    if (cached) return res.json(JSON.parse(cached));
  }
  const result = await createRoute(req.body);
  if (key) await redis.setex(`idem:${key}`, 86400, JSON.stringify(result));
  res.status(201).json(result);
});

// Rate Limiting Algorithms:
// Token Bucket: N tokens, refill R/sec. Allows bursts.
// Sliding Window: count requests in [now-W, now]. Most accurate.
// Fixed Window: count per time bucket. Simple but edge-bursts.
```

Interview Questions

Interview Question	Answer
API versioning?	URL path /api/v1/ (recommended). Only bump for BREAKING changes. Additive changes are backward-compatible.
Cursor vs offset pagination?	Offset: O(n) for large skips, inconsistent with concurrent writes. Cursor: O(log n), consistent, production standard.
Idempotency?	Same request = same result. Use Idempotency-Key header, cache response in Redis. Critical for payment APIs.

BUILD TASK
(1) Full REST CRUD with JSON envelope, (2) Cursor pagination, (3) Idempotency middleware, (4) OpenAPI auto-generation.

Chapter 10: WebSockets & Real-Time

NEXUS Relevance: *NEXUS Dashboard streams live metrics via Socket.io.*

Code & Implementation

```
const { Server } = require('socket.io');

function setupWS(httpServer) {
  const io = new Server(httpServer, { cors: { origin: '*' } });

  // Auth on connect
  io.use((socket, next) => {
    try {
      socket.user = jwt.verify(socket.handshake.auth.token, SECRET);
      next();
    } catch { next(new Error('Auth failed')); }
  });

  io.of('/dashboard').on('connection', (socket) => {
    socket.join(`tenant:${socket.user.sub}`);
  });

  // Stream metrics every second
  setInterval(async () => {
    const metrics = await collectMetrics();
    io.of('/dashboard').emit('metrics:global', metrics);
  }, 1000);
  return io;
}
```

Interview Questions

Interview Question	Answer
WebSocket vs SSE?	WS: bidirectional, full-duplex. SSE: server→client only, auto-reconnect, HTTP-based. WS for chat/collab, SSE for notifications/dashboards.
Scaling WebSockets?	Redis adapter: all servers share events via Redis Pub/Sub. Or sticky sessions. Socket.io has built-in Redis adapter.

BUILD TASK
(1) Socket.io with /dashboard namespace, (2) JWT auth on connection, (3) Tenant-based rooms, (4) Live metrics streaming.

Chapter 11: Microservices & Circuit Breaker

NEXUS Relevance: *NEXUS is an API Gateway — the core microservices pattern.*

Code & Implementation

```
class CircuitBreaker {
  constructor(opts={}) {
    this.threshold = opts.failureThreshold || 5;
    this.resetTimeout = opts.resetTimeout || 10000;
    this.state = 'CLOSED'; this.failures = 0;
    this.lastFailure = null;
  }
  async execute(fn, fallback) {
    if (this.state === 'OPEN') {
      if (Date.now()-this.lastFailure > this.resetTimeout)
        this.state = 'HALF_OPEN';
      else return fallback ? fallback() : Promise.reject('Circuit OPEN');
    }
    try {
      const result = await fn();
      this.failures = 0;
      if (this.state === 'HALF_OPEN') this.state = 'CLOSED';
      return result;
    } catch (e) {
      this.failures++; this.lastFailure = Date.now();
      if (this.failures >= this.threshold) this.state = 'OPEN';
      if (fallback) return fallback();
      throw e;
    }
  }
}

// Per-upstream circuit breakers
const breakers = new Map();
async function proxy(upstream, req) {
  if (!breakers.has(upstream))
    breakers.set(upstream, new CircuitBreaker());
  return breakers.get(upstream).execute(
    () => forwardRequest(upstream, req),
    () => ({ status: 503, body: 'Service unavailable' })
  );
}
```

Interview Questions

Interview Question	Answer
Monolith vs Microservices?	Start monolith. Split when: team >5, independent scaling needed, different deploy frequencies. Don't split: small team, unclear boundaries, cross-service transactions needed.
Circuit breaker?	3 states: Closed (normal), Open (fail-fast after threshold), Half-Open (probe after timeout). Prevents cascading failures.

BUILD TASK
(1) CircuitBreaker class, (2) Per-upstream breakers, (3) Health endpoint showing circuit states, (4) Fallback responses.

Chapter 12: Message Queues — BullMQ & Pub/Sub

NEXUS Relevance: *NEXUS uses BullMQ for async log processing and Redis Pub/Sub for real-time config propagation.*

Code & Implementation

```
const { Queue, Worker } = require('bullmq');

const logQueue = new Queue('request-logs', {
  connection: new Redis(process.env.REDIS_URL),
});

// Producer: async log processing
async function logRequest(data) {
  await logQueue.add('process', data, {
    attempts: 3,
    backoff: { type: 'exponential', delay: 1000 },
  });
}

// Consumer: background worker
const worker = new Worker('request-logs', async (job) => {
  await db.query(
    'INSERT INTO request_logs (route_id,status_code,latency_ms) VALUES ($1,$2,$3)',
    [job.data.routeId, job.data.statusCode, job.data.latencyMs]
  );
}, { connection: new Redis() });

worker.on('failed', (job, err) => {
  if (job.attemptsMade >= job.opts.attempts)
    logger.error('Job DLQ:', job.id, err);
});
```

Interview Questions

Interview Question	Answer
Queue vs Pub/Sub?	Queue: guaranteed delivery, persistence, one consumer, retries. Pub/Sub: fire-and-forget, all subscribers, no persistence. Queue for tasks that MUST complete, Pub/Sub for real-time events.
Dead letter queue?	Where messages go after all retries fail. Without DLQ: failed messages lost forever. Always monitor DLQ depth.

BUILD TASK
(1) BullMQ for log processing, (2) Redis Pub/Sub for config broadcasting, (3) DLQ with alerting, (4) Scheduled hourly aggregation job.

Chapter 13: Docker & Containerization

NEXUS Relevance: *NEXUS runs in optimized Docker containers with multi-stage builds.*

Code & Implementation

```
# Multi-stage Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
RUN addgroup -g 1001 nexus && adduser -u 1001 -G nexus -s /bin/sh -D nexus
COPY --from=builder /app/package*.json ./
RUN npm ci --omit=dev
COPY --from=builder /app/dist ./dist
HEALTHCHECK --interval=30s CMD wget -qO- http://localhost:3000/health || exit 1
USER nexus
EXPOSE 3000
CMD ["node", "dist/index.js"]

# docker-compose.yml
# services:
#   nexus: { build: ., ports: ['3000:3000'], depends_on: [postgres, redis] }
#   postgres: { image: postgres:16-alpine, volumes: ['pgdata:/var/lib/postgresql/data'] }
#   redis: { image: redis:7-alpine }
```

Interview Questions

Interview Question	Answer
Optimize Node Docker image?	Multi-stage build, node:alpine (5x smaller), COPY package*.json first (layer cache), npm ci --omit=dev, non-root user, .dockerignore. Result: 100-150MB vs 1GB+.
Docker Compose vs K8s?	Compose: local dev, single host. K8s: production, multi-host, auto-scaling, self-healing. Compose for dev, K8s for production at scale.

BUILD TASK
(1) Multi-stage Dockerfile, (2) docker-compose with gateway+PostgreSQL+Redis, (3) HEALTHCHECK, (4) Non-root user, (5) Volume for PostgreSQL.

Chapter 14: Testing — Unit, Integration, E2E

NEXUS Relevance: *NEXUS has comprehensive tests: unit (Jest), integration (Supertest), load (autocannon).*

Code & Implementation

```
// Unit test: CircuitBreaker
describe('CircuitBreaker', () => {
  test('opens after threshold failures', async () => {
    const breaker = new CircuitBreaker({ failureThreshold: 3 });
    const fail = () => Promise.reject(new Error());
    for (let i = 0; i < 3; i++)
      await breaker.execute(fail, () => 'fallback');
    expect(breaker.state).toBe('OPEN');
  });
});

// Integration test: API endpoints
const request = require('supertest');
describe('POST /api/v1/routes', () => {
  test('creates route', async () => {
    const res = await request(app)
      .post('/api/v1/routes')
      .set('Authorization', `Bearer ${token}`)
      .send({ path: '/test', upstream: 'http://svc:3001' });
    expect(res.status).toBe(201);
  });
  test('rejects invalid data', async () => {
    const res = await request(app)
      .post('/api/v1/routes')
      .set('Authorization', `Bearer ${token}`)
      .send({ path: '/test', upstream: 'not-a-url' });
    expect(res.status).toBe(422);
  });
});
```

Interview Questions

Interview Question	Answer
Unit vs Integration vs E2E?	Unit: single function, mocked deps, fast. Integration: multiple components together (API→DB). E2E: full user flow. Ratio: 70/20/10 (testing pyramid).
Test database strategy?	Docker container with test DB, reset between tests. Or mock DB layer for unit tests. Never test against production.

BUILD TASK
(1) Unit: CircuitBreaker, rate limiter, JWT utils, (2) Integration: auth + route CRUD endpoints, (3) E2E: register→login→createRoute→proxy, (4) Load test: 1000 RPS for 30s with autocannon.

Chapter 15: Security — OWASP Top 10

NEXUS Relevance: *NEXUS is a security boundary protecting upstream services.*

Code & Implementation

```
const helmet = require('helmet');
app.use(helmet()); // 11 security headers
app.use(express.json({ limit: '10mb' })); // prevent DoS

// SQL injection prevention: ALWAYS parameterize
const result = await db.query('SELECT * FROM users WHERE id=$1', [id]);
// NEVER: db.query(`SELECT * FROM users WHERE id='${id}'`);

// XSS prevention
const xss = require('xss');
const safe = xss(req.body.content); // strips <script> tags

// CORS
app.use(cors({
  origin: ['https://dashboard.nexus.com'],
  credentials: true,
  methods: ['GET','POST','PATCH','DELETE'],
}));

// Rate limiting
const rateLimit = require('express-rate-limit');
app.use('/api/', rateLimit({ windowMs: 15*60*1000, max: 100 }));
app.use('/auth/', rateLimit({ windowMs: 15*60*1000, max: 10 }));
```

Interview Questions

Interview Question	Answer
OWASP Top 10?	(1) Broken Access Control, (2) Crypto Failures, (3) Injection, (4) Insecure Design, (5) Misconfiguration, (6) Vulnerable Components, (7) Auth Failures, (8) Integrity Failures, (9) Logging Failures, (10) SSRF. Every backend engineer must know all 10.
Prevent SQL injection?	Parameterized queries ($1,$2). Never concatenate user input into SQL. ORM does this automatically. Also: validate input types, least-privilege DB user.

BUILD TASK
(1) Helmet.js, (2) Input validation everywhere, (3) SQL injection audit, (4) CORS whitelist, (5) Per-API-key + per-IP rate limiting, (6) Audit log for admin actions.

Chapter 16: Performance & Caching

NEXUS Relevance: *NEXUS targets 10K+ RPS with multi-layer caching.*

Code & Implementation

```
// 3-LAYER CACHE: LRU → Redis → Database
const LRU = require('lru-cache');
const localCache = new LRU({ max: 1000, ttl: 60000 });

async function getRoute(path) {
  let r = localCache.get(path);
  if (r) return r;
  const cached = await redis.get(`route:${path}`);
  if (cached) { r = JSON.parse(cached); localCache.set(path, r); return r; }
  r = await prisma.route.findFirst({ where: { path, active: true } });
  if (r) {
    await redis.setex(`route:${path}`, 300, JSON.stringify(r));
    localCache.set(path, r);
  }
  return r;
}

// ETag middleware
function etag(req, res, next) {
  const origJson = res.json.bind(res);
  res.json = (body) => {
    const tag = crypto.createHash('md5').update(JSON.stringify(body)).digest('hex');
    res.setHeader('ETag', `"${tag}"`);
    if (req.headers['if-none-match'] === `"${tag}"`) return res.status(304).end();
    return origJson(body);
  };
  next();
}

// Compression
const compression = require('compression');
app.use(compression({ threshold: 1024 }));
```

Interview Questions

Interview Question	Answer
Caching strategy?	L1: in-process LRU (1-10ms). L2: Redis (1-5ms). L3: database (5-50ms). Cache-aside pattern. Invalidate on write via Redis Pub/Sub. TTL based on data freshness.
ETag?	Hash of response. Client sends If-None-Match. If same, server returns 304 No Content (no body). Saves bandwidth.

BUILD TASK
(1) 3-layer cache, (2) ETag middleware, (3) Gzip compression, (4) Connection pooling for upstreams, (5) Profile with clinic.js.

Chapter 17: Logging & Observability

NEXUS Relevance: *NEXUS uses structured logging, distributed tracing, and Prometheus metrics.*

Code & Implementation

```
const winston = require('winston');
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [new winston.transports.Console()],
});

// Request ID (distributed tracing)
const { v4: uuid } = require('uuid');
app.use((req, res, next) => {
  req.requestId = req.headers['x-request-id'] || uuid();
  res.setHeader('X-Request-ID', req.requestId);
  next();
});

// Prometheus metrics
const client = require('prom-client');
const httpReqs = new client.Counter({
  name: 'nexus_requests_total',
  help: 'Total requests',
  labelNames: ['method','path','status'],
});
const httpDur = new client.Histogram({
  name: 'nexus_request_duration_seconds',
  help: 'Request duration',
  buckets: [0.01, 0.05, 0.1, 0.5, 1, 5],
});
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
});
```

Interview Questions

Interview Question	Answer
Structured vs unstructured logging?	Unstructured: console.log('error happened'). Structured: logger.error({event:'route_fail',routeId:'abc'}). JSON output, searchable, filterable. Always structured in production.
Distributed tracing?	Follow request across services via X-Request-ID. All services log with this ID. Debug by searching traceId. Tools: Jaeger, Datadog APM.

BUILD TASK
(1) Winston structured logger, (2) X-Request-ID middleware, (3) Prometheus counter + histogram, (4) /metrics endpoint.

Chapter 18: CI/CD & Deployment

NEXUS Relevance: *NEXUS uses GitHub Actions for CI and blue-green Docker deployment.*

Code & Implementation

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16-alpine, env: { POSTGRES_DB: test, POSTGRES_PASSWORD: test },
ports: ['5432:5432'] }
      redis: { image: redis:7-alpine, ports: ['6379:6379'] }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run lint
      - run: npm run test -- --coverage
      - run: npm run build
  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    steps:
      - run: docker build -t nexus:${{ github.sha }} .
      - run: docker push registry/nexus:latest
```

Interview Questions

Interview Question	Answer
Blue-green vs Canary vs Rolling?	Blue-green: two envs, swap traffic. Instant rollback. 2x cost. Canary: 5%→100% traffic. Safest. Rolling: update one-by-one. No extra infra but mixed versions.
CI/CD pipeline?	CI: lint→typecheck→unit tests→integration tests→build→security scan. CD: Docker build→push→deploy staging→smoke tests→approval→deploy prod→health check.

BUILD TASK
(1) GitHub Actions: lint→test→build→Docker, (2) PostgreSQL+Redis services in CI, (3) Health check gate, (4) Rollback on failure.

Chapter 19: System Design — Classic Problems

NEXUS Relevance: *System design is asked in every senior interview. NEXUS itself IS a system design project.*

Code & Implementation

```
// URL SHORTENER: base62(auto_increment_id), Redis cache, 301 redirect
// RATE LIMITER: sliding window, Redis sorted sets, X-RateLimit headers
// CHAT SYSTEM: WebSocket + Redis adapter, PostgreSQL messages, presence via Redis
// NOTIFICATION: event→queue→workers (FCM/SES/Twilio), user preferences
// API GATEWAY: NEXUS! routing, auth, rate limit, circuit break, monitoring
// DISTRIBUTED CACHE: consistent hashing, replication, eviction policies
// FILE UPLOAD: presigned S3 URLs, multipart, virus scan, metadata in DB
// TASK SCHEDULER: delay queue, cron, Redis sorted sets by execution time
// SEARCH: inverted index, Elasticsearch, autocomplete with trie/prefix
// ANALYTICS: event streaming (Kafka), OLAP (ClickHouse), real-time dashboards
```

Interview Questions

Interview Question	Answer
Design URL shortener?	POST /shorten, GET /:code→301. Generate: base62(auto_increment). Cache: Redis. Analytics: async queue. Scale: shard by code hash. 7 chars = 3.5T URLs.
Design chat system?	WebSocket + Redis adapter. PostgreSQL for messages. Redis for presence. Partition by conversation_id. Push notifications for offline.
Design notification system?	Event→priority queue→workers per channel (push/email/SMS). User preferences, templates, dedup, rate limiting. DLQ for failures.

BUILD TASK
Design on paper (30 min each): URL Shortener, Rate Limiter, Chat, Notifications, API Gateway, Distributed Cache, File Upload, Task Scheduler, Search, Analytics Dashboard.

Chapter 20: AI Integration — LLMs, RAG, Embeddings

NEXUS Relevance: *NEXUS AI Layer: auto-docs, anomaly detection, natural language queries.*

Code & Implementation

```
const OpenAI = require('openai');
const openai = new OpenAI();

// Auto-generate API docs from logs
async function generateDocs(routePath) {
  const logs = await prisma.requestLog.findMany({
    where: { path: routePath }, take: 50 });
  const res = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [{ role: 'system', content: 'Generate OpenAPI docs from these logs.' },
      { role: 'user', content: JSON.stringify(logs.slice(0,10)) }],
  });
  return res.choices[0].message.content;
}

// Natural language → SQL
async function nlToQuery(question) {
  const res = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [{ role: 'system', content: `Convert to PostgreSQL. Schema:
request_logs(route_id,status_code,latency_ms,created_at). Return ONLY SQL.` },
      { role: 'user', content: question }],
  });
  const sql = res.choices[0].message.content.trim();
  if (!sql.toUpperCase().startsWith('SELECT')) throw new Error('Only SELECT allowed');
  return prisma.$queryRawUnsafe(sql);
}
```

Interview Questions

Interview Question	Answer
LLM in backend?	API call to OpenAI. Patterns: structured output (prompt for JSON), RAG (retrieve+augment), tool use, streaming. Always: timeout, cache, validate output, never trust for security.
RAG?	Retrieval-Augmented Generation: embed question→search vector DB→include context in prompt→LLM answers grounded in your data. Prevents hallucination.

BUILD TASK
(1) Auto-doc generation from logs, (2) Anomaly detection with embeddings, (3) NL→SQL interface, (4) Worker Thread for AI calls, (5) Response caching.

Chapter 21: Advanced — Cluster, Worker Threads

NEXUS Relevance: *NEXUS scales via Cluster (multi-core) and Worker Threads (CPU-intensive AI).*

Code & Implementation

```
const cluster = require('cluster');
const os = require('os');

if (cluster.isPrimary) {
  for (let i = 0; i < os.cpus().length; i++) cluster.fork();
  cluster.on('exit', (w, code) => {
    console.log(`Worker ${w.process.pid} died`);
    if (code !== 0) cluster.fork();
  });
  // Zero-downtime restart on SIGUSR2
  process.on('SIGUSR2', () => {
    const workers = Object.values(cluster.workers);
    function restart(i) {
      if (i >= workers.length) return;
      workers[i].disconnect();
      workers[i].on('disconnect', () => {
        cluster.fork().on('listening', () => restart(i+1));
      });
    }
    restart(0);
  });
} else {
  require('./app').listen(3000);
  process.on('SIGTERM', () => {
    server.close(() => process.exit(0)); // graceful
  });
}
```

Interview Questions

Interview Question	Answer
Cluster vs Worker Threads?	Cluster: multiple processes, separate memory, share port. For HTTP scaling. Worker Threads: threads in one process, share memory (SharedArrayBuffer). For CPU-intensive tasks. NEXUS: Cluster for requests, Workers for AI.
Zero-downtime deploys?	Rolling restart: disconnect worker→fork new→wait listening→repeat. At no point zero workers running. Or blue-green with load balancer.

BUILD TASK
(1) Cluster mode per CPU core, (2) Worker Threads for AI, (3) Graceful shutdown on SIGTERM, (4) Rolling restart on SIGUSR2.

Chapter 22: NEXUS Assembly — The Final Build

NEXUS Relevance: *Connect all 21 chapters into the complete NEXUS system.*

Code & Implementation

```
// NEXUS Final Architecture:
// Load Balancer → NEXUS Cluster (N workers)
//   → Middleware: RequestID → Auth → RateLimit → Cache
//     → CircuitBreaker → Proxy → Logger
//   → Plugin Engine (dynamic middleware)
//   → Dashboard (Socket.io + Redis adapter)
//   → AI Layer (Worker Threads + OpenAI)
// Infrastructure: PostgreSQL + Redis + BullMQ Workers

// INTERVIEW ANSWER: 'Tell me about NEXUS'
// NEXUS is an AI-powered distributed API Gateway I built from scratch.
// It handles: dynamic routing, JWT+OAuth2+API key auth, RBAC,
// sliding-window rate limiting (Redis sorted sets), per-upstream
// circuit breaking with fallback, real-time WebSocket dashboard,
// AI-powered auto-documentation + anomaly detection + NL queries,
// running in cluster mode with Docker, CI/CD, and Prometheus metrics.
// Handles 10K+ RPS, p99 <50ms for cached routes.
```

Interview Questions

Interview Question	Answer
Walk me through NEXUS.	AI-powered API Gateway built from scratch. Dynamic routing, JWT+OAuth2+API key auth, RBAC, sliding-window rate limiting, circuit breaking, WebSocket dashboard, AI features (auto-docs, anomaly detection, NL→SQL). Node.js, Express, PostgreSQL, Redis, BullMQ, Socket.io, OpenAI, Docker, GitHub Actions. 10K+ RPS, p99 <50ms.
Hardest part?	Distributed cache invalidation across cluster workers. Solved with Redis Pub/Sub: route update publishes event, all workers subscribe and invalidate local LRU cache. Tested with 4 workers, verified consistency within 1 second.

BUILD TASK

FINAL: (1) Connect ALL subsystems, (2) README with architecture diagram, (3) Demo video, (4) Deploy on AWS with Docker, (5) Blog post about architecture. THIS IS YOUR PORTFOLIO CENTERPIECE.

Your 90-Day Execution Plan

Days	DSA (4-5h)	Backend Skills (4-5h)	NEXUS Project (1-2h)
1-10	Arrays, Strings, Hashing	Ch 1-3: Node internals, Async, Modules	Project setup, raw HTTP server, monorepo
11-20	Binary Search, Sliding Window	Ch 4-5: HTTP deep dive, Express	Express foundation, middleware chain, validation
21-30	Two Pointers, Stack	Ch 6: Auth (JWT, OAuth2, RBAC)	Auth server: registration, login, tokens, RBAC
31-40	Trees, BST	Ch 7-8: Databases, Prisma	Database layer: PostgreSQL, Redis, migrations
41-50	Graphs (BFS, DFS, Dijkstra)	Ch 9-10: API design, WebSockets	REST API, dashboard WebSocket streaming
51-60	Dynamic Programming	Ch 11-12: Microservices, Queues	Circuit breaker, message queue, pub/sub
61-70	Backtracking, Heap	Ch 13-15: Docker, Testing, Security	Dockerize, write tests, security hardening
71-80	Mixed practice + contests	Ch 16-18: Performance, Monitoring, CI/CD	Caching, observability, CI/CD pipeline
81-90	Mock interviews + weak areas	Ch 19-22: System Design, AI, Assembly	AI layer, final assembly, deploy, document

THE RESULT

After 90 days, you will have: (1) 300+ DSA problems solved by pattern, (2) Production-grade Node.js backend knowledge across 22 topics, (3) NEXUS — a project that no other fresher on the planet has, (4) Interview readiness for any company worldwide. This is not a dream. This is a plan with daily actions. Execute it.